Drogoe Dorinica - Stefana
324 CA

Examen

2) $T(n) = 3T\left(\frac{n}{2}\right) + 2T\left(\frac{n}{2}\right) + 2048 \cdot n^2 - \log^2 n$

$T(n) = 5T\left(\frac{n}{2}\right) + 2048 n^2 - \log^2 n$

$a = 5 \quad b = 2 \quad f(n) = 2048 n^2 - \log^2 n$

$n^{\log_2 5} \boxed{?} 2048 n^2 - \log^2 n$

$n^{2,\ldots} \boxed{?} 2048 n^2 - \log^2 n$

Vreau să $f(n) = O(n^{\log_2 5 - \varepsilon})$

$$L = \lim_{n \to \infty} \frac{2048 n^2 - \log^2 n}{n^{\log_2 5 - \varepsilon}} = \lim_{n \to \infty} 2048 \cdot n^{2 - \log_2 5 + \varepsilon} - \lim_{n \to \infty} \frac{\log^2 n}{n^{\log_2 5 - \varepsilon}}$$

L trebuie $\in [0, \infty)$

$$\begin{cases} \log_2 5 - \varepsilon \geq 0 \Rightarrow \log_2 5 \geq \varepsilon \\ 2 - \log_2 5 + \varepsilon \leq 0 \Rightarrow \varepsilon \leq \log_2 5 - 2 \end{cases}$$

Deci pentru $\varepsilon \in [0; \log_2 5 - 2]$

$L = 0 - 0 = 0 \Rightarrow f(n) = O(n^{\log_2 5 - \varepsilon})$

Aplic cazul I de Master $\Rightarrow$

$T(n) = \Theta(n^{\log_2 5})$

Scanned with CamScanner

3) Cazul de bază: se aplică pentru constructori nulari și externi

↳ BT(e, tr))

P tree = empty

Ne propunem să dem că:

P(empty): memberBT(e, empty) == member(e, toList(empty))

MB MD

cf. (MB) => MS = false

cf. (T1) => MD = member(e, [])

cf. (M1) => MD = false

=> MS == MD == false => P(empty) adev.

Ipoteza inductivă: P(tree)

P(tree): memberBT(e, tree) == member(e, toList(tree))

(∀) l e ET tree ∈ BTree<T>

Pasul de inducție: Considerăm P(tree) adevărată

dem că P(node(left, x, right)) adev.

P(tree) → P(node(left, x, right))

P(node(left, x, right)):

MD

memberBT(e, node(left, x, right)) == member(e, toList(node(left, x, right)))

MS

cf. (MB2) =>

MS = (e == x)

memberBT(e, tr))

|| memberBT(e, l) ||

Scanned with CamScanner

$\mathcal{G}(T_2) \Rightarrow$

MD = member(e, (tolist(l)) ++ (x : tolist(r)))

Boco ((e == x || memberBT(e, l) || memberBT(e, r))

false $\Rightarrow$

M.S. false $\Rightarrow$ P(tnode(l, x, r)) true

Deci { e == x true sau
memberBT(e, l) true sau
memberBT(e, r) true

Cum P(tree) true $\Rightarrow$

{ e == x true sau
memberBT(e, tolist(l)) true
memberBT(e, tolist(r)) true

Am nevoie de proprietatea:

$P_1(e)$: member(e, l_1 ++ l_2) = member(e, l_1) || member(e, l_2)

M.D. $\overset{P_1}{=}$ member(e, tolist(l)) || member(e, x : tolist(r))

$\mathcal{G}\ M_2 \Rightarrow$

MD = membol(e, tolist(l)) || membol(e, tolist(r)) || (e == x)

$\Rightarrow$ M.S = MD $\Rightarrow$ P(tree) true

Mai avem de dem doar că:

P(e) true

Scanned with CamScanner

Caz de bază:

$P_1([]) : member(e, [] ++ \ell_2) == member(e, []) \,||\, member(e, \ell_2)$

cf. $A_1 \Rightarrow member(e, \ell_2) ==$ false $||\, member(e, \ell_2)$

$M_1 \Rightarrow P_1([])$ true

Ip. ind: $P_1(\ell)$ true

Pas: $P_1(\ell) \rightarrow P_1(x:\ell)$ true

$P_1(x:\ell)$: member $(e, (x:\ell) ++ \ell_2) ==$ member $(e, x:\ell) \,||$ member (e, ℓ_2)

cf. A_2 și $M_2 \Rightarrow$

member $(e, x:(\ell ++ \ell_2)) ==$ ~~member e~~ $(e == x) \,||$ member $(e, \ell) \,||$ member (e, ℓ_2)

cf. $M_2 \Rightarrow$

~~member~~ $(e == x) \,||$ member $(e, (\ell ++ \ell_2)) == (e = x) \,||$ member $(e, \ell) \,||$ member (e, ℓ_2)

Ader cf $P_1(\ell) \Rightarrow$

$P_1(e:\ell)$ ader. $\Rightarrow$

P (free) ader.

Scanned with CamScanner

1) N secv. ADN

cele N secv. au un șir de dim K

```
function solve(N, K)
    șir = ∅
    for (i = 1 .. n)
        construct_ADN = choice(A, C, G, T)
        șir = șir.append(construct_ADN);
        // dacă am depășit dimensiunea subșirului
        if (i > K)
            // caut fiecare literă
            for (j = 1 .. n)
                if (! find(secv[j], șir))
                    fail
            succes
    fail
```

Funcția care găsește dacă ~~##~~ caractere din s_1 apar în s_2

```
function find(s1, s2)
    for each car in s2
        count[car]++;
    for each value in count
        if (value != 0)
            return false
    return true
```

Complexitate: $O(n^3)$

Scanned with CamScanner

```
N-ADN (strings k, n, sizes)
    for (i=1 __ n)                                   O(n·k)
        v[i] = ∅
        if (sizes[i] < k)
            fail
        for (j=1 __ k)
            pas = choice{1 __ sizes[i]}
            if (last(v[i]) == null || last[i] < pas)
                add last(v[i])
            else
                fail
    for (i=1 __ k)                                   O(n·k)
        for (j=1 __ N-1)
            if (strings[j][v[j][i]] == strings[j+1]
                                       [v[j+1][i]])
                fail
```

$\Rightarrow O(n \cdot k)$

Scanned with CamScanner